

openclaw/imshow: structure and algorithms

Research report R1b, input to the media and MCP platform redesign

Subject: github.com/openclaw/imshow, commit `d604a8be548a637faa35fe968af426c7484ded67` (2026-10-06, version 0.15.10)
Compared against: `EQStack apps/imshow-mcp` at `91b08bb` (read-only)
Plan this serves: `docs/plans/2026-10-08-mcp-platform-and-media.md` (plan worktree)
Diagram tool: `Typst @preview/fletcher 0.5.8` (`d2`, `dot` and `mmmc` are not installed on this machine)

Citation convention. Every claim is `path:line @ sha`. Their paths are repo-relative to `openclaw/imshow` and carry `#sha` (short form of the full SHA above). Our paths are relative to `apps/imshow-mcp/` and carry `#our`. “Unknown” means I did not find evidence; it never means “no”.

1 Executive summary

The first finding changes the brief. `openclaw/imshow` is a Swift CLI plus a JSON-RPC-over-stdio server plus an injected Objective-C dylib. It is **not** an MCP server, and it has **no** OCR, no speech-to-text, no vision calls, no interpretation cache and no MIME inference. A repo-wide search of `Sources/**/*.swift` for `vision`, `VNRecognize`, `SFSpeech`, `transcri`, `whisper` and `OCR` (case-insensitive) finds exactly two things: the reader of Apple’s **stored** voice-memo transcript (`Sources/IMsgCore/MessageStore+Attachments.swift:20-97 @ d604a8b`) and an unrelated AppleScript UI element name (`Sources/imshow/Commands/ReactCommand.swift:203 @ d604a8b`). A search for MCP, JSON Schema and `openapi` finds only prose in docs. So of our seven named structural problems, `openclaw/imshow` gives direct answers to **three** (god file, three-frameworks/drift, honest pagination as a side effect), partial answers to **three** (dates, MIME, plugin payloads), and **nothing** for two (single interpretation slot, cache provenance). Where it has nothing, I say so rather than stretch.

It is still valuable, mainly as a very carefully engineered **data-access and transport** codebase: 22,219 lines of Swift in `Sources/imshow` and `Sources/IMsgCore`, 26,595 lines of tests with 816 `@Test` cases, 7,280 lines of Objective-C helper (counts from `find/wc` over the clone). Its strengths are boring in the good way: capability probing instead of version sniffing, one row-decoding path, typed delivery failures, bounded everything.

1.1 Scorecard against our known problems

Our problem	What openclaw/imshow does	Value
Untyped Apple-epoch dates	Converts at one boundary (<code>appleEpoch(Date)</code> and <code>appleDate(from:)</code> , <code>Sources/IMsgCore/MessageStore+Helpers.swift:39-55 @ d604a8b</code>); user dates parsed by a parser that throws (<code>Sources/IMsgCore/MessageFilter.swift:14-26 @ d604a8b</code>). But it assumes every column is nanoseconds and never reads <code>attachment.created_date</code> , so it dodges our unit bug rather than solving it.	Shape yes, unit model no
MIME/kind logic copied 5x	No MIME inference at all: passes <code>mime_type</code> through (<code>Sources/IMsgCore/MessageStore+Attachments.swift:49-60 @ d604a8b</code>). Its one classification, the CAF/GIF conversion plan, OR-s three signals (UTI, extension, MIME) in one place (<code>Sources/IMsgCore/AttachmentResolver.swift:149-174 @ d604a8b</code>).	Pattern yes, code no
Plugin-payload filtering	Treats plugin payloads as message rows identified by <code>balloon_bundle_id</code> and coalesces URL previews into the preceding text row (<code>Sources/IMsgCore/MessageStore+URLPreviews.swift:13-98 @ d604a8b</code>). Read side has no filter for <code>.pluginPayloadAttachment</code> attachment rows; that suffix appears only on the send side (<code>Sources/imshow/RichLinkPreparer+LinkPresentation.swift:23 @ d604a8b</code>).	A new signal for our predicate
One interpretation slot per message	Not present. No interpretation concept exists.	None
Cache with no provenance	Not present for interpretations. Its only cache (converted audio/GIF) is content-addressed by path, size and mtime (<code>Sources/IMsgCore/AttachmentResolver.swift:62-78 @ d604a8b</code>).	Weak hint only
2892-line god file	Largest <code>IMsgCore</code> source file is 500 lines; the store is 23 <code>MessageStore+*.swift</code> extensions over a shared schema struct, row-selection builder and query structs (<code>Sources/IMsgCore/MessageStoreSchema.swift:3-53 @ d604a8b</code> , <code>Sources/IMsgCore/MessageStore+MessageRows.swift:94-152 @ d604a8b</code>).	High, directly reusable design
Three MCP frameworks	No MCP. Its RPC layer is one descriptor table driving dispatch, lane scheduling, capability gating and the advertised method list (<code>Sources/imshow/</code>	High, maps onto <code>mcp-kit v3</code>

Our problem	What openclaw/imshow does	Value
	RPCMethodDescriptors.swift:56-247 @ d604a8b), with fail-closed parameter parsing (Sources/imshow/RPCParameters.swift:4-51 @ d604a8b).	

1.2 Top 10 structural lessons, ranked by impact on our known problems

- Split the store by concern around three shared objects, not one class.** A capability struct built once from PRAGMA table_info (Sources/IMsgCore/MessageStoreSchema.swift:24-53 @ d604a8b), a MessageRowSelection that builds the SELECT list and substitutes literal NULL for absent columns (Sources/IMsgCore/MessageStore+MessageRows.swift:103-149 @ d604a8b), and one decodeMessageRow (Sources/IMsgCore/MessageStore+Messages.swift:359-421 @ d604a8b). Every query is a small struct that owns its SQL and bindings (Sources/IMsgCore/MessageStore+Queries.swift:12-135 @ d604a8b). This is the template for dismantling imessage-db.ts (src/imessage-db.ts:1-2892 @ 91b08bb).
- One descriptor table is the single source of truth for a tool/method.** Names plus aliases, route, execution lane, database requirement and bridge requirement live in one record (Sources/imshow/RPCMethodDescriptors.swift:56-105 @ d604a8b); the same table filters the advertised list so only **usable** methods are listed (Sources/imshow/RPCMethodDescriptors.swift:232-237 @ d604a8b, Sources/imshow/RPCServer+StatusHandlers.swift:140 @ d604a8b). Feed this shape into the mcp-kit v3 registry: it removes the two-definitions-per-tool drift and makes tools/list capability-aware.
- Read Apple's own voice-memo transcript from attachment.user_info.** They parse the binary plist and take key audio-transcription (Sources/IMsgCore/MessageStore+Attachments.swift:67-101 @ d604a8b), then override message text with it (Sources/IMsgCore/MessageStore+Messages.swift:385-388 @ d604a8b). We read a **different** location, the IMAudioTranscription attribute inside attributedBody (src/imessage-db.ts:2088 @ 91b08bb, src/attribution-body-text.ts:119 @ 91b08bb), and grep finds no user_info or audio-transcription anywhere in our src/. A second free, local, non-LLM source in front of any model call is the cheapest fix for the "CAF transcript is a cached refusal" bug.
- Type time at the boundary, and make parsing throw.** One Date in the model, one converter pair, MessageFilter.fromISO throws invalidISODate rather than returning nil (Sources/IMsgCore/MessageFilter.swift:14-26 @ d604a8b). Caveat: they assume nanoseconds everywhere (Sources/IMsgCore/MessageStore+Helpers.swift:41,54 @ d604a8b). Our branded-unit plan (AppleNanos/AppleSeconds) is strictly more correct than theirs, so take the **shape**, not the unit model.
- Classify media by agreeing signals in one place, and convert behind a content-addressed, atomic cache.** uti == "com.apple.coreaudio-format" || .caf || audio/x-caf in one predicate (Sources/IMsgCore/AttachmentResolver.swift:157-160 @ d604a8b); output name = prefix + SHA-256(path|size|mtime) (Sources/IMsgCore/AttachmentResolver.swift:62-78 @ d604a8b); write to a hidden UUID temp file then move (Sources/IMsgCore/AttachmentResolver.swift:105-120 @ d604a8b); optional converter, graceful omission (Sources/IMsgCore/AttachmentResolver.swift:92-94 @ d604a8b); report converted_* beside the original instead of replacing it (Sources/IMsgCore/AttachmentResolver.swift:42-53 @ d604a8b).
- Plugin payloads are a property of the message row.** balloon_bundle_id identifies the plugin (Sources/IMsgCore/MessageStore+URLPreviews.swift:10,96-98 @ d604a8b); the preview is folded into the preceding text row only if same chat, direction, sender and handle, a later ROWID, within 5 seconds, and the text contains the URL (Sources/IMsgCore/MessageStore+URLPreviews.swift:74-94 @ d604a8b). Add message.balloon_bundle_id IS NOT NULL to our plugin-payload predicate alongside the filename/UTI signals.
- Make pagination honest by construction.** Fetch limit + 1, trim, and return a nextRowID cursor with hasMore (Sources/IMsgCore/MessageStore+Messages.swift:127-199 @ d604a8b); when coalescing shrinks a page, double the physical limit and retry (Sources/IMsgCore/MessageStore+Messages.swift:77-80,65-73 @ d604a8b). Ours is messages.length === limit (src/index.ts:665 @ 91b08bb), which is the heuristic the plan's B item wants to remove.
- Give mutations a typed delivery disposition and have the script report its own phase.** DeliveryFailure carries notStarted | mayHaveCompleted | stillInFlight and retrySafe (Sources/IMsgCore/DeliveryFailure.swift:4-44 @ d604a8b); the AppleScript sets dispatchPhase before the first send and returns a tab-separated IMSG_RESULT line (Sources/IMsgCore/MessageSender.swift:210-275 @ d604a8b); SMS fallback happens only when retrySafe (Sources/IMsgCore/MessageSender.swift:196-198 @ d604a8b); a stillInFlight result poisons the mutation lane (Sources/imshow/RPCServer+Runtime.swift:126-141 @ d604a8b). Ours does a post-hoc chat.db read (src/delivery-status.ts:1-40 @ 91b08bb) and a 30 s execFile timeout (src/applescript.ts:72-75 @ 91b08bb), which are complementary, not equivalent.
- Watcher hygiene we are missing.** Re-arm on inode change (Sources/IMsgCore/MessageWatcher.swift:106-116,186-209 @ d604a8b), a bounded stream whose overflow error carries resumeAfterRowID (Sources/IMsgCore/MessageWatcher.swift:38-44,292-298 @ d604a8b), and a retry loop (limit 20) for a message row that appears before its chat_message_join row (Sources/IMsgCore/

MessageWatcher.swift:86,335-353 @ d604a8b). I found no equivalent of the last one in our watcher (src/change-watcher.ts:1-297 @ 91b08bb); treat as **verify**.

10. **Subprocess and path hardening for the media pipeline.** Monotonic deadline, SIGTERM then SIGKILL of child and process group (Sources/IMsgCore/ProcessTimeout.swift:18-60 @ d604a8b), openat with O_NOFOLLOW per component and a regular-file check (Sources/IMsgCore/AttachmentSource.swift:10-50 @ d604a8b), and exclusive-create staging (Sources/IMsgCore/AttachmentSource.swift:52-82 @ d604a8b). Our media helpers shell out to sips/qlmanage/afinfo (src/media.ts:1-17 @ 91b08bb); the planned afconvert/ffmpeg stage should inherit these controls. Same family: the build recipe for a signed, notarised Swift helper with an embedded Info.plist for TCC identity (Package.swift:42-52 @ d604a8b, scripts/build-universal.sh:62-78 @ d604a8b), which is what packages/apple/intel needs.

2 Architecture of openclaw/ormsg

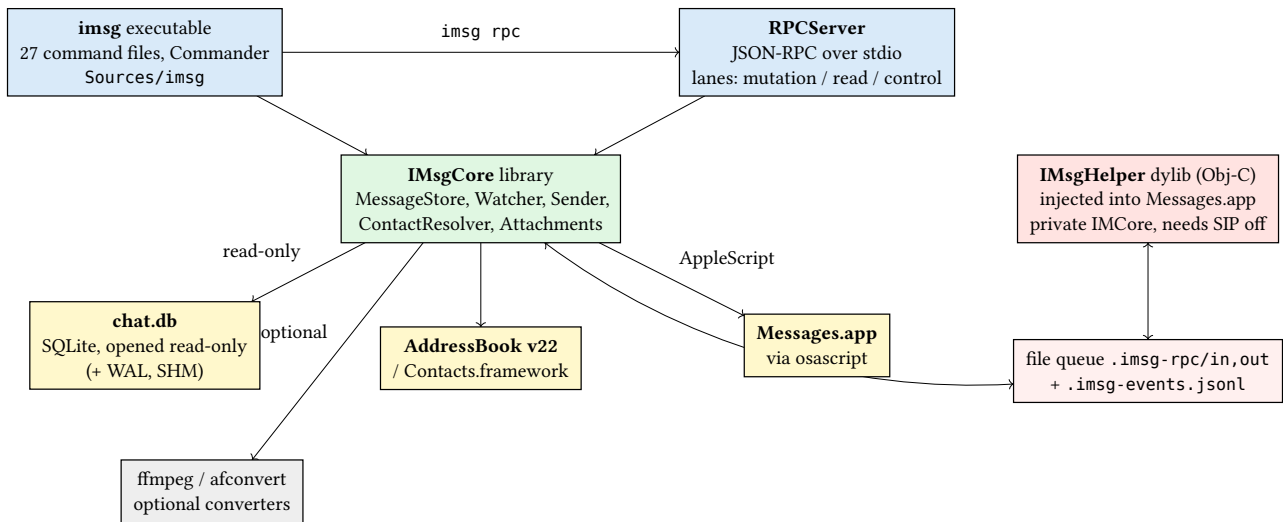


Figure 1: openclaw/ormsg module graph. Swift package targets from Package.swift:5-53 @ d604a8b the helper from Sources/IMsgHelper/README.md:3 @ d604a8b the queue protocol from Sources/IMsgCore/IMsgBridgeProtocol.swift:3-23 @ d604a8b SIP requirement from README.md:89 @ d604a8b.

The package has two Swift targets and one non-Swift target. IMsgCore (library, 11,240 lines) depends on SQLite.swift, CSQLite (Linux only) and PhoneNumberKit, and links ScriptingBridge and Contacts (Package.swift:20-30 @ d604a8b). ormsg (executable, 10,979 lines) depends on IMsgCore and Commander (Package.swift:33-37 @ d604a8b). IMsgHelper is a single Objective-C translation unit that #includes 31 .inc fragments (Sources/IMsgHelper/README.md:3 @ d604a8b), compiled by clang into a dylib (Makefile:62-76 @ d604a8b). There is no dependency on any network or ML framework.

3 Per-area analysis

3.1 Module layout and dependency graph

Theirs	Library/executable split with the library owning all behaviour (Package.swift:8-9 @ d604a8b). Within IMsgCore, one MessageStore class (149 lines) is extended by 23 files named MessageStore+Concern.swift (Attachments, Chats, Messages, Polls, Reactions, Search, Stats, URLPreviews, Scheduled, and so on). The class itself holds only the connection, a serial queue and the schema flags (Sources/IMsgCore/MessageStore.swift:4-87 @ d604a8b). Strongly typed ids exist as MessageID, ChatID, HandleID (Sources/IMsgCore/DatabaseIDs.swift:1-11 @ d604a8b). Platform seams use #if os(macOS) inside files rather than parallel trees, so Linux compiles the same read core (TestsLinux/ @ d604a8b).
Ours	One class, IMessageDB, 2892 lines (src/imessage-db.ts:1-2892 @ 91b08bb); Tables constants and epoch helpers in db-schema.ts (src/db-schema.ts:13-22 @ 91b08bb). No id types.
Verdict	ADOPT the extension-per-concern layout and the three shared objects (schema, row selection, row decode). Do not copy their file count mechanically: Swift extensions share private state for free, whereas TypeScript needs explicit modules and an injected context.
Lands in	A new package packages/apple/chatdb (proposed) holding schema.ts (capability probe), row-selection.ts, decode-row.ts, and queries/*.ts (one file per use case). apps/ormsg-mcp/src/imessage-db.ts shrinks to a facade. Branded MessageID/ChatID/HandleID types go in the same package.

3.2 chat.db access: queries, schema versions, WAL and locking, read-only safety

Theirs	Opens through a file; URI with mode=ro and readonly: true (Sources/IMsgCore/MessageStore.swift:25-27 @ d604a8b), busyTimeout = 5 seconds (Sources/IMsgCore/MessageStore.swift:28 @ d604a8b). All access serialised on one DispatchQueue with a re-entrancy check (Sources/IMsgCore/MessageStore.swift:79-86 @ d604a8b). No version sniffing. Capabilities are probed from PRAGMA table_info for four tables into 19 booleans (Sources/IMsgCore/MessageStoreSchema.swift:24-53 @ d604a8b, Sources/IMsgCore/MessageStore+Helpers.swift:12-21 @ d604a8b). SQL is built with schema.hasX ? "m.x" : "NULL" (Sources/IMsgCore/MessageStore+MessageRows.swift:106-127 @ d604a8b); features degrade rather than fail, for example unreadOnly against a schema without is_read is rejected explicitly (Sources/IMsgCore/MessageStore+Chats.swift:159-165 @ d604a8b). Open errors (out of memory (14), authorization denied, unable to open) are mapped to a typed permissionDenied with Full Disk Access instructions (Sources/IMsgCore/MessageStore+Helpers.swift:29-37 @ d604a8b, Sources/IMsgCore/Errors.swift:17-34 @ d604a8b). A custom SQLite function msg_search_text does Unicode-aware matching inside SQL (Sources/IMsgCore/MessageStore+Search.swift:17-38 @ d604a8b); search then re-checks in Swift and grows the physical limit until the page fills (Sources/IMsgCore/MessageStore+Search.swift:80-140 @ d604a8b). ROWID cursors are physical: for multi-chat messages the chat id is the MIN(chat_id) subselect (Sources/IMsgCore/MessageStore+MessageRows.swift:95-97 @ d604a8b). The RPC layer owns the database through an actor that re-validates file identity (device and inode) and re-opens when the file is replaced, never caching a failed open as terminal (Sources/IMsgCore/RPCDatabaseResources.swift:70-140 @ d604a8b). Linux read mode uses a copy produced by SQLite's backup command, not cp (docs/linux.md:47-53 @ d604a8b). No WAL pragma, no immutable=1; WAL is only watched, not configured (Sources/IMsgCore/MessageWatcher.swift:168-170 @ d604a8b).
Ours	new Database(path, { readonly: true }) (src/ibase-db.ts:227 @ 91b08bb); busy_timeout = 5000 set because node:sqlite defaults to 0 (src/sqlite.ts:128-132 @ 91b08bb). Column presence checked lazily with PRAGMA table_info per call site (src/ibase-db.ts:2308 @ 91b08bb), scattered rather than held in one struct. Permission errors: access-check.ts (151 lines).
Verdict	ADAPT the probe-once capability struct and the NULL-substitution SQL builder; ADAPT the file-identity re-open for long-lived MCP processes (their own RPC docs say a replaced or restored database path must rotate to a fresh generation, docs/rpc.md:77-88 @ d604a8b). Our readonly open is equivalent to their second flag; whether node:sqlite accepts a mode=ro URI is unknown.
Lands in	packages/apple/chatdb/schema.ts (probeSchema(db) -> ChatDbCapabilities) and open.ts (identity-checked reopen). Capability object is also surfaced to the mcp-kit registry (lesson 2).

3.3 Date and epoch handling, and units

Theirs	A single constant appleEpochOffset = 978_307_200 (Sources/IMsgCore/MessageStore.swift:5 @ d604a8b). Bind side: appleEpoch(Date) -> Binding multiplies by 1e9, rejects non-finite, and falls back to a REAL binding when Int64 would overflow so a valid date is never clipped (Sources/IMsgCore/MessageStore+Helpers.swift:39-49 @ d604a8b). Read side: appleDate(from:) divides by 1e9 and maps null to the epoch (Sources/IMsgCore/MessageStore+Helpers.swift:51-55 @ d604a8b). The domain model carries Date, not integers (Sources/IMsgCore/MessageStore+MessageRows.swift:67 @ d604a8b). Unit assumption: nanoseconds, for message.date, date_read and every comparison (Sources/IMsgCore/MessageStore+Messages.swift:370,418-419 @ d604a8b). The code never selects attachment.created_date, which is where seconds appear, so the bug class cannot arise (Tests/IMsgCoreTests/MessageDatabaseFixture.swift:102-111 @ d604a8b shows the fixture attachment table has no such column; a repo-wide search for created_date returns nothing). User input: ISO8601Parser tries fractional then plain internet date-time (Sources/IMsgCore/ISO8601.swift:4-14 @ d604a8b) and MessageFilter.fromISO throws on an unparseable string (Sources/IMsgCore/MessageFilter.swift:17-24 @ d604a8b). Relative phrases like "3 days ago" are not supported by the core; DurationParser.swift exists in the CLI (unknown scope, not read).
Ours	Two converters: macTimestampToDate assumes ns; macAutoTimestampToDate auto-detects seconds vs ns at a 1e15 boundary (src/db-schema.ts:59-86 @ 91b08bb), while the header comment still says all

	timestamps are nanoseconds (src/db-schema.ts:10 @ 91b08bb). parseUserData builds local time, rejects seconds and Z, and returns null (src/date-parse.ts:29-40 @ 91b08bb). The plan counts about 8 scattered conversions (plan, section “What’s structurally wrong”, item 7).
Verdict	WE ARE AHEAD on unit awareness (their code would also mishandle attachment.created_date). ADOPT the failure semantics (throw, never silent null; REAL-binding fallback on overflow) and the “domain model holds a real date type” rule.
Lands in	packages/time/apple: branded AppleNanos/AppleSeconds, toBinding() with the overflow fallback, and a per-column unit table. packages/time/user-date: return Result, keep our richer grammar. Add a property test: round-trip Date -> binding -> Date across 2001..2100 and across the int64 edge.

3.4 Attachment model: MIME, UTI and extension inference, plugin payloads, stickers, link previews, CAF voice memos

Theirs	Metadata is a straight projection of filename, transfer_name, uti, mime_type, total_bytes, is_sticker (Sources/IMsgCore/MessageStore+Attachments.swift:4-18 @ d604a8b) plus original_path, missing, and optional converted_path/converted_mime_type (Sources/IMsgCore/AttachmentResolver.swift:42-53 @ d604a8b). The docs describe mime_type as “best-effort MIME from UTI” (docs/attachments.md:26 @ d604a8b) but the code does no inference; null stays null. Conversion plan. A record of target extension, MIME and ffmpeg argv, selected by OR-ing UTI, path suffix and MIME: CAF to M4A (aac 128k), GIF to first-frame PNG (Sources/IMsgCore/AttachmentResolver.swift:14-18,149-174 @ d604a8b). Output is cached under the user caches dir, named from a sanitised stem plus SHA-256 of path size mtime, written atomically (Sources/IMsgCore/AttachmentResolver.swift:62-78,105-120,176-184 @ d604a8b). A Linux build falls back to FNV-1a because CryptoKit is absent (Sources/IMsgCore/AttachmentResolver.swift:186-201 @ d604a8b). CAF voice memos on receive: Apple’s transcript from attachment.user_info plist key audio-transcription (Sources/IMsgCore/MessageStore+Attachments.swift:83-101 @ d604a8b), gated on is_audio_message (Sources/IMsgCore/MessageStore+Messages.swift:386 @ d604a8b). On send: afconvert -f caff -d opus@24000 -b 32000 -c 1, then validated with AudioToolbox that the result is mono 24 kHz Opus with a finite non-zero duration (Sources/IMsgCore/AudioMessagePreparer.swift:42-75 @ d604a8b). Stickers: read side is only the is_sticker flag (Sources/IMsgCore/MessageStore+Attachments.swift:11 @ d604a8b); the send side validates size 1..512000 bytes, pixel counts and frame counts with ImageIO, and checks PNG/GIF container completeness (Sources/IMsgCore/StickerAsset.swift:37-60,359-384 @ d604a8b). Link previews and plugin payloads: see lesson 6. Row-level, keyed on balloon_bundle_id. NOT IN THEIR REPO: any read-side exclusion of .pluginPayloadAttachment attachment rows.
Ours	kindFromMime (mime, then extension) in media-intel-runtime.ts (src/media-intel-runtime.ts:84-91 @ 91b08bb), audioFormat in media-providers.ts (src/media-providers.ts:141 @ 91b08bb), two TUI copies and two SQL fragments (plan item 8). Apple transcript comes from attributedBody only (src/imessage-db.ts:2088 @ 91b08bb). No user_info read anywhere in src/.
Verdict	ADOPT : (a) the second Apple-transcript source; (b) the multi-signal predicate shape; (c) hashed, atomic conversion cache with sibling converted_* fields; (d) balloon_bundle_id as an extra plugin signal; (e) the afconvert Opus recipe and AudioToolbox post-check for the send side. SKIP their lack of MIME inference: that is exactly our bug.
Lands in	packages/media/kind: classify({uti, mime, ext, balloonBundleId, isSticker}) -> {kind, mime, isPluginPayload} plus a kindSqlFragment() that mirrors it; golden table tests. packages/media/intel: sources/apple-transcript.ts (tries attributedBody then user_info), convert/ stage with the sha-keyed atomic cache. packages/apple/chatdb: attachments.ts selects user_info when hasAttachmentUserInfo.

3.5 attributedBody and typedstream decoding

Theirs	63 lines. Handles three shapes: UTF-16LE with BOM ff fe; typedstream with header 04 0b and the signature streamtyped (little-endian) or typedstream (big-endian); anything else decoded as UTF-8 with leading control characters trimmed (Sources/IMsgCore/TypedStreamParser.swift:4-28 @ d604a8b). For typedstream it scans for the first 01 2b (+, the C-string/NSString content tag) and reads a length: a single
---------------	--

	byte, or 0x81/0x82 followed by a 2- or 4-byte integer in the archive’s byte order; it rejects the reserved range 0x80..0x91 and any length overrun (Sources/IMsgCore/TypedStreamParser.swift:30-52 @ d604a8b). Framing by length means embedded marker bytes in the text cannot truncate it. Tests cover plain, length-prefixed and UTF-16 bodies (Tests/IMsgCoreTests/MessageStoreAttributedBodyTests.swift:7-200 @ d604a8b).
Ours	Two implementations (Rust native and a TS parser) with a “does the result look trustworthy” filter of regexes for attachment GUIDs, __KIM, NS* class names and a doubled-capital heuristic for leaked length bytes (src/attributed-body-text.ts:18-37 @ 91b08bb), a METADATA_PATTERNS blocklist (src/attributed-body-text.ts:39-66 @ 91b08bb), a 200 ms parse timeout (src/attributed-body-text.ts:74 @ 91b08bb), and a preamble matcher for 01 94 95 84 01 2b (src/parsers/typedstream-parser.ts:26-38 @ 91b08bb).
Verdict	ADAPT . Their deterministic length-framed read after the first 01 2b is smaller and has no heuristic filter. It is a candidate primary path with ours as fallback, but only after a differential test over a corpus (synthetic plus our existing fixtures): the first 01 2b may not be the message body in every layout (for example messages whose first string attribute precedes the body). Whether it is has not been verified here, so it is unknown .
Lands in	packages/apple/chatdb/decode/typedstream.ts: port of parseAttributedBody as strict; keep the heuristic parser as lenient; a test harness that runs both over the fixture set and fails on disagreement.

3.6 Contacts and identity resolution

Theirs	A protocol ContactResolving with displayName(for:), displayNames(for:), searchByName (Sources/IMsgCore/ContactResolver.swift:23-31 @ d604a8b). The catalog is a process-owned snapshot with a state machine over authorization (authorized addressBook notDetermined unavailable restricted), invalidation by CNContactStoreDidChange, a 30 s TTL, a “last good catalog” flag so a transient failure does not blank names, and bounded per-region snapshots (max 8, LRU) (Sources/IMsgCore/ContactCatalog.swift:5-13, 99-196, 198-237 @ d604a8b, Sources/IMsgCore/ContactResolver.swift:91-106 @ d604a8b). Headless stdin uses skipIfNotDetermined so a LaunchAgent never blocks on a prompt that cannot resolve (Sources/IMsgCore/ContactResolver.swift:53-58 @ d604a8b). Over SSH it reads AddressBook-v22 SQLite directly, in place, refusing any version other than 22 so stale migrations never serve names, and distinguishing busy from unavailable (Sources/IMsgCore/AddressBookContacts.swift:17-18, 46, 56-63 @ d604a8b). Matching heuristics are minimal: phone numbers through PhoneNumberKit to E.164 per region, otherwise the input is returned unchanged (Sources/IMsgCore/PhoneNumberNormalizer.swift:7-14 @ d604a8b); email lowercased; chat-guid prefixes (iMessage;-; , SMS;+; , any;-; and so on) stripped before lookup (Sources/IMsgCore/ContactCatalog.swift:244-251 @ d604a8b); first contact wins on a collision (Sources/IMsgCore/ContactCatalog.swift:212, 215 @ d604a8b); name search is a case-insensitive substring returning the first phone or email (Sources/IMsgCore/ContactCatalog.swift:73-90 @ d604a8b). Sender fallback to destination_caller_id when the handle is empty (Sources/IMsgCore/MessageStore+Messages.swift:390-393 @ d604a8b). No persistent identity, no slugs, no merge across handles.
Ours	Three explicit normal forms (MATCH, KEY, SEND) with golden pins and a documented slug-stability constraint (src/handle-normal.ts:1-31 @ 91b08bb), contact ranking (contact-rank.ts), vCard comparison, persistent slugs (slug-store.ts, thread-slug.ts). Contacts are read from our own contacts-db.ts.
Verdict	WE ARE AHEAD on identity and matching; theirs is deliberately simple. ADOPT only two operational ideas: the authorization state machine with “last good catalog” semantics, and the headless “never prompt” policy. ADAPT their AddressBook version gate (refuse versions other than the one validated) if we read AddressBook directly.
Lands in	packages/apple/contacts (proposed, small) for the catalog refresh/auth state machine, only if contacts-db.ts is extracted. Otherwise a note in contacts-db.ts. No change to handle-normal.ts.

3.7 Send pipeline: AppleScript and private API, retries, error detection

Theirs	Two transports. AppleScript via /usr/bin/osascript -l AppleScript - with the script on stdin and arguments in argv; stdout/stderr captured to 0600 files in a 0700 temp dir; hard timeout of 150 s (Sources/IMsgCore/AppleScriptSendTransport.swift:8-84 @ d604a8b, Sources/IMsgCore/IMsgBridgeProtocol.swift:30 @ d604a8b). The script tracks dispatchPhase (pre_dispatch then
---------------	--

	dispatch_started) and always returns <code>IMSG_RESULT<TAB>ok failure<TAB>phase<TAB>errno</code>; a missing or malformed result line is conservatively <code>mayHaveCompleted</code> (Sources/IMsgCore/MessageSender.swift:210-275 @ d604a8b, Sources/IMsgCore/AppleScriptSendTransport.swift:86-106 @ d604a8b). Injected bridge via IMCore private APIs through a file queue (v2: atomic <uuid>.tmp rename into in/, response in out/; v1 was a single overwritten file that raced) (Sources/IMsgCore/IMsgBridgeProtocol.swift:3-11 @ d604a8b). Routing: attachments are staged by copying into <code>~/Library/Messages/Attachments/img/<uuid>/</code> through the safe-open path (Sources/IMsgCore/MessageSender.swift:123-166 @ d604a8b). SMS fallback is allowed only when the user asked for auto, the target is a phone number, the message is text-only, and the failure is <code>retrySafe</code> (Sources/IMsgCore/MessageSender.swift:99-106,196-198 @ d604a8b). Failure classification funnels through one classifier that maps pre-publication errors to <code>notStarted</code> and post-publication errors to <code>mayHaveCompleted</code> for mutations (Sources/IMsgCore/BridgeFailureClassifier.swift:4-34 @ d604a8b). Mutations are never retried implicitly: <code>isMutation</code> is a property of the action enum (Sources/IMsgCore/IMsgBridgeProtocol.swift:99-107 @ d604a8b). Verification after send reads <code>chat.db</code> for a matching sent row (SendMessageVerifier.swift, 155 lines, not read in detail; and Sources/IMsgCore/MessageStore+AttachmentReceipts.swift:5-46 @ d604a8b for attachments by staged path).
Ours	<code>execFile("osascript", ["-e", script])</code> with 30 s timeout, stderr redacted before logging (src/applescript.ts:72-83 @ 91b08bb); delivery truth derived afterwards from <code>error</code>, <code>is_sent</code>, <code>is_finished</code>, <code>was_downgraded</code> (src/delivery-status.ts:1-40 @ 91b08bb). No retry-safety notion. I found no <code>dispatchPhase</code>-style signal in <code>applescript.ts</code>.
Verdict	ADOPT the phase protocol and the Disposition vocabulary; keep our <code>chat.db</code> delivery-truth derivation as the second stage. SKIP the injected bridge (requires SIP disabled, README.md:89 @ d604a8b). The 150 s timeout is empirically derived for macOS 26 (Sources/IMsgCore/IMsgBridgeProtocol.swift:28-30 @ d604a8b); our 30 s is likely too short on that OS, so verify before copying the number.
Lands in	<code>packages/apple/send</code> (proposed): <code>script.applescript</code> as a string constant with the phase protocol, <code>interpret(stdout) -> Result, Disposition</code> type, <code>retrySafe</code> getter; <code>apps/img-mcp</code> send tool surfaces the disposition in <code>structuredContent</code>. Pair with the <code>mcp-kit</code> mutation lane (next area).

3.8 Watching for new messages: FSEvents versus polling

Theirs	DispatchSource vnode sources (kqueue, not FSEvents) on three files, <code>chat.db</code>, <code>-wal</code> and <code>-shm</code>, plus one on the directory, each with an inode/device identity so a replaced file is re-armed (Sources/IMsgCore/MessageWatcher.swift:106-116,168-170,186-215 @ d604a8b). Events are debounced 0.25 s, with a 5 s fallback poll that also refreshes registrations (Sources/IMsgCore/MessageWatcher.swift:7-31,243-266 @ d604a8b). The cursor is a ROWID high-water mark, never file state; each poll drains in bounded batches (default 100) and re-schedules itself while the cursor advances (Sources/IMsgCore/MessageWatcher.swift:268-315 @ d604a8b). Output is an <code>AsyncThrowingStream</code> with <code>bufferingOldest(256)</code>; a dropped yield ends the stream with <code>MessageWatcherOverflowError(resumeAfterRowID:)</code> so the client can resume with no gap (Sources/IMsgCore/MessageWatcher.swift:38-44,66,292-298 @ d604a8b). A message row without a resolved chat is retried up to 20 times then skipped (Sources/IMsgCore/MessageWatcher.swift:335-353 @ d604a8b). URL-preview rows are de-duplicated with a 90 s window and 10 minute retention (Sources/IMsgCore/MessageStore.swift:89-125 @ d604a8b). Helper events are tailed from a JSONL file with rotation handling (Sources/IMsgCore/IMsgEventTailer.swift:35-39 @ d604a8b).
Ours	Wal file watch, directory watch, 10 s safety poll, 150 ms debounce, <code>maxBatch</code> pages, never throws into the host, documented live finding that the directory watch alone delivered nothing on the real Messages directory (src/change-watcher.ts:1-38 @ 91b08bb).
Verdict	WE ARE AHEAD overall, and our write-up of the TCC/FSEvents blind spot is better evidence than theirs. ADOPT three details: watch <code>-shm</code> too, overflow-with-resume-cursor for any bounded subscriber stream, and the unresolved-chat retry (verify we have an equivalent; I did not find one).
Lands in	<code>packages/apple/chatdb/watch.ts</code> once the watcher is extracted from <code>change-watcher.ts</code>; the resume-cursor error type belongs in the shared event contract used by <code>wait_for_changes</code>.

3.9 Media: transcription, OCR, conversion, caching

Theirs	NOT IN THEIR REPO: OCR, speech recognition, vision captioning, any model call. Media handling is limited to (1) reading Apple's stored transcript, (2) optional ffmpeg conversion for CAF and GIF (docs/
--------	--

	attachments.md:36-52 @ d604a8b), (3) afconvert for outgoing voice notes (Sources/IMsgCore/AudioMessagePreparer.swift:42-56 @ d604a8b), (4) ImageIO validation of stickers (Sources/IMsgCore/StickerAsset.swift:1-60 @ d604a8b), (5) LinkPresentation for rich-link metadata on send (Sources/IMsgCore/RichLinkPreparer+LinkPresentation.swift:30 @ d604a8b). Converter processes are bounded by the 60 s ProcessTimeout (Sources/IMsgCore/AttachmentResolver.swift:129,131-147 @ d604a8b).
Ours	media-intel*.ts, media-providers.ts, media.ts: provider chain, cache, runtime; zero-dependency sips/qlmanage/mdls/afinfo helpers (src/media.ts:1-17 @ 91b08bb).
Verdict	Nothing to copy for engines or caching. ADOPT only the conversion stage mechanics (lesson 5) and the process controls (lesson 10). Engine design must come from R2/L, not from this repo.
Lands in	packages/media/intel/convert/(stage), packages/os/proc (proposed, tiny: runBounded(argv, {timeoutMs, killGroup})).

3.10 Caching and persistence

Theirs	Only on-disk state in IMsgCore is the conversion cache under Library/Caches/IMsgCore/converted-attachments (Sources/IMsgCore/AttachmentResolver.swift:176-184 @ d604a8b), staged send copies under Library/Messages/Attachments/IMsgCore/ which are explicitly not pruned (docs/attachments.md:66 @ d604a8b), and in-process caches: reply-parent memo keyed by guid that caches misses as well as hits (Sources/IMsgCore/MessageStore+ReplyContext.swift:4-16 @ d604a8b), poll option text cache, contact snapshots. No database of its own, no logs on disk (stderr only; a verbose flag, Sources/IMsgCore/RuntimeOptions.swift:5-10 @ d604a8b). Persistent slugs and identities: unknown/none.
Ours	slugs.db, interpretation cache (media-intel-cache.ts, 183 lines), analytics cache, structured logging to files (logger.ts).
Verdict	WE ARE AHEAD . One transferable micro-idea: negative caching of lookups that failed, scoped to one query loop (Sources/IMsgCore/MessageStore+ReplyContext.swift:12-15 @ d604a8b). Do not apply it to interpretation results.
Lands in	No new package. Note in media-intel v2 design that failures need their own status and TTL, not negative-by-omission.

3.11 Concurrency model

Theirs	Swift 6 strict concurrency. Data access: serial queues. RPC: an actor RPCScheduler with three lanes : all mutations run through one FIFO worker; reads run up to four in parallel; control requests have their own worker; admission limited to 128 outstanding requests (Sources/IMsgCore/RPCServer+Runtime.swift:23-70 @ d604a8b). A mutation includes validation, staging, bridge call, the response and post-send verification before the next mutation starts (docs/rpc.md:101-106 @ d604a8b). If a mutation ends stillInFlight, the lane is poisoned : queued and future mutations are rejected with mutationLaneBlocked until restart (Sources/IMsgCore/RPCServer+Runtime.swift:126-141 @ d604a8b). Launch of Messages.app is guarded by a serial coordinator plus a cross-process lock file whose path is checked for symlinks (Sources/IMsgCore/BridgeLaunchCoordinator.swift:7-80 @ d604a8b). Stdin is read by one dedicated thread (Sources/IMsgCore/RPCServer+Runtime.swift:7-20 @ d604a8b).
Ours	Node single thread; node:sqlite synchronous; watchdog (watchdog.ts), event bus, shutdown registry (shutdown.ts). No lane concept: unknown whether concurrent MCP send_message calls are serialised.
Verdict	ADOPT the mutation lane and poison-on-ambiguous-failure: two concurrent sends from an agent loop are the realistic duplicate-send risk. Reads need no cap beyond what the host already imposes.
Lands in	mcp-kit v3 dispatcher option lane: "mutation" "read" "control" declared in the descriptor (lesson 2); the poison signal is raised by packages/apple/send (previous area).

3.12 Error-handling types

Theirs	A closed IMsgError enum with user-oriented descriptions (Sources/IMsgCore/Errors.swift:3-13 @ d604a8b); DeliveryFailure for mutations (Sources/IMsgCore/DeliveryFailure.swift:24-67 @ d604a8b), which sanitises detail to one line and 512 characters (Sources/IMsgCore/DeliveryFailure.swift:58-66 @ d604a8b); separate typed errors for the launcher, tailer, stickers and polls. The RPC boundary maps errors with one function: caller-caused (invalidISODate, invalidChatTarget ...) become JSON-RPC invalidParams, everything else internalError; DeliveryFailure has its own error shape (Sources/IMsgCore/RPCServer+Dispatch.swift:107-131,160-169 @ d604a8b). Cancellation is swallowed quietly (Sources/IMsgCore/RPCServer+Dispatch.swift:107 @
---------------	---

	d604a8b). Warning: the rule “callers must never infer retry safety by matching description” is written into the type’s doc comment (Sources/IMsgCore/DeliveryFailure.swift:20-23 @ d604a8b).
Ours	Mixed Error/string errors; MCP errors wrapped by home-made dispatchers (plan item 1). Results for interpretation lack a closed status type (plan item 3).
Verdict	ADOPT the two-class mapping (caller-caused vs internal) at the single tool boundary, and the “typed field, never string matching” rule. It is the same discipline the plan’s InterpretResult (ok refused failed unsupported) needs.
Lands in	mcp-kit v3: ToolError with kind: "invalid_params" "internal" "delivery" and retrySafe?. packages/media/intel: InterpretResult as planned.

3.13 Testing and fixtures: how they fake chat.db

Theirs	Swift Testing (@Test), 816 cases (counted by <code>grep -c</code>). Fixtures are built in code: an in-memory SQLite connection with CREATE TABLE statements parameterised by a SchemaOptions struct that toggles optional columns (attributedBody, reaction columns, audio flag, balloon, payload data, scheduled, read state, chat routing), emulating older macOS schemas (Tests/IMsgCoreTests/MessageDatabaseFixture.swift:25-48 @ d604a8b). The store has a test initialiser that accepts the capability flags directly (Sources/IMsgCore/MessageStore.swift:36-77 @ d604a8b). Every external effect is an injected closure on RPCServer (send, bridge invoke, stagers, clock, stream providers) (Sources/Imsg/RPCServer.swift:50-120 @ d604a8b), so RPC tests drive handleLineForTesting with raw JSON strings, including a battery of malformed-parameter requests that must each yield -32602 (Tests/ImsgTests/RPCContractValidationTests.swift:7-60 @ d604a8b). Native tests for the Obj-C helper compile real fragments into small host binaries (Makefile:32-47 @ d604a8b). Binary fixtures are synthetic and documented (a 0.5 s 440 Hz tone stored as base64, Tests/IMsgCoreTests/Fixtures/README.md:1-12 @ d604a8b). A Linux CI job runs the read-core tests plus a standalone-archive smoke test in a bare Ubuntu container (scripts/check-linux.sh:11-32 @ d604a8b, TestsLinux/LinuxReadCoreTests.swift @ d604a8b); the docs tell users to copy a live database with SQLite’s .backup rather than cp (docs/linux.md:47-53 @ d604a8b). Weakness shared with us: fixtures use nanosecond dates and always populate mime_type (Tests/ImsgTests/CommandTestDatabase.swift:7-10,41-50 @ d604a8b), so their fixtures too could not catch a seconds-vs-nanoseconds or null-MIME bug.
Ours	scripts/generate-fixtures.ts generates a file DB with nanosecond created_date, no null-MIME rows and no plugin payloads (plan item 9, citing scripts/generate-fixtures.ts:702-806).
Verdict	ADOPT the SchemaOptions matrix : generate each fixture under several column-presence profiles and run the same assertions; and the malformed-parameter battery for tool schemas. Add what neither repo has: realistic-value rows (seconds created_date, null MIME, plugin payloads). ADOPT differential testing against their built imsg binary on our synthetic fixture as an oracle for decoding (optional, see Reuse).
Lands in	packages/apple/chatdb/testing/fixture.ts (buildFixture(profile)), shared by imsg-mcp tests and the media packages.

3.14 Packaging and distribution: signing, notarisation, Homebrew

Theirs	One version source version.env (version.env:1 @ d604a8b) generates Version.swift, Info.plist and a C header (scripts/generate-version.sh:1-30 @ d604a8b). scripts/build-universal.sh builds per-arch with swift build --arch, merges with lipo, builds the dylib with a required slice list (arm64e arm64 t8020, failing if one is missing because macOS 26 silently refuses an arm64-only dylib), and signs with the Developer ID identity using --timestamp --options runtime and explicit --identifier (scripts/build-universal.sh:11-16,37-78 @ d604a8b). Ad-hoc signing (-) for dev. Entitlements: Apple Events and Address Book only (Resources/Imsg.entitlements:5-8 @ d604a8b). The CLI embeds an Info.plist into the binary at link time via -sectcreate __TEXT __info_plist, giving a bare executable a bundle identifier and the NSAppleEventsUsageDescription / NSContactsUsageDescription strings, which is what lets TCC prompts name it (Package.swift:46-51 @ d604a8b, Sources/Imsg/Resources/Info.plist:1-20 @ d604a8b). Releases use a shared reusable workflow openclaw/release-workflows@v1 with notarisation, checksum inventory, independent verifier jobs and a Homebrew tap handoff (docs/RELEASING.md:3-22 @ d604a8b). A Linux static-runtime tarball is built too. CI pins action SHAs and runs lint, tests, a no-send native probe and a build (.github/workflows/ci.yml:16-60 @ d604a8b). Vendored-dependency patching is scripted (scripts/patch-deps.sh:1-30 @ d604a8b).
---------------	---

Ours	npm package built by Vite; Rust .node addon prebuilt for darwin-arm64 only (native/imshow-native.darwin-arm64.node @ 91b08bb); manifest.json for MCPB; release by multi-semantic-release (repo CLAUDE.md).
Verdict	ADOPT the recipe for packages/apple/intel: -sectcreate Info.plist for TCC identity (Speech Recognition and Vision usage strings are required for the permission prompts), lipo universal, codesign --options runtime --identifier, a build step that asserts required slices. SKIP Homebrew/Linux for now; we ship through npm and MCPB.
Lands in	packages/apple/intel/ (Swift, per plan) with scripts/build.sh, Resources/Info.plist, Resources/intel.entitlements; notarisation is out of scope until a distribution decision is made (unknown requirement).

3.15 MCP layer: schema generation and JSON Schema dialect

Theirs	NOT IN THEIR REPO for MCP and JSON Schema. The contract is JSON-RPC 2.0 over stdio with hand-maintained docs tables (docs/json.md:1-40 @ d604a8b, docs/rpc.md:1-60 @ d604a8b). Parameters are parsed from [String: Any] by a hand-written accessor that (a) rejects any key not in the method's supported set, (b) is type-strict (a string "2" is not an integer, true is not 1, Sources/imshow/RPCParameters.swift:18-51 @ d604a8b, docs/rpc.md:22-26 @ d604a8b), (c) allows only explicit, per-method aliases (docs/rpc.md:28-44 @ d604a8b), and (d) requires exactly one of chat_id chat_identifier chat_guid (Sources/imshow/RPCParameters.swift:85-91 @ d604a8b). A status/initialize response advertises rpcUsableMethods filtered by live database and bridge capability (Sources/imshow/RPCServer+StatusHandlers.swift:140 @ d604a8b). Descriptor table: Sources/imshow/RPCMethodDescriptors.swift:107-223 @ d604a8b. No output schemas exist; outputs are dictionaries built by hand (Sources/imshow/OutputModels.swift:1-60 @ d604a8b, not read in full: unknown whether any are Codable structs).
Ours	Two hand-written definitions per tool (JSON in mcp-tools.ts, zod in mcp-schemas.ts), three zod-to-JSON converters across repos, draft-07 output rejected by a client update (plan "Context" and item 1).
Verdict	Nothing about dialects. ADOPT only the behavioural contract : fail-closed unknown keys, strict types, explicit aliases, mutually-exclusive selector groups expressed in the schema (oneOf with required), capability-gated listing. In zod 4 this is z.strictObject plus z.xor-style refinements, emitted to 2020-12 by the kit converter.
Lands in	@george43g/mcp-kit v3: defineTool({ name, aliases, input, output, lane, requires: { db: [...], bridge: [...] } }); listTools(caps) filters on requires; conformance helper includes a "reject unknown key" case per tool.

4 Their structure to our packages

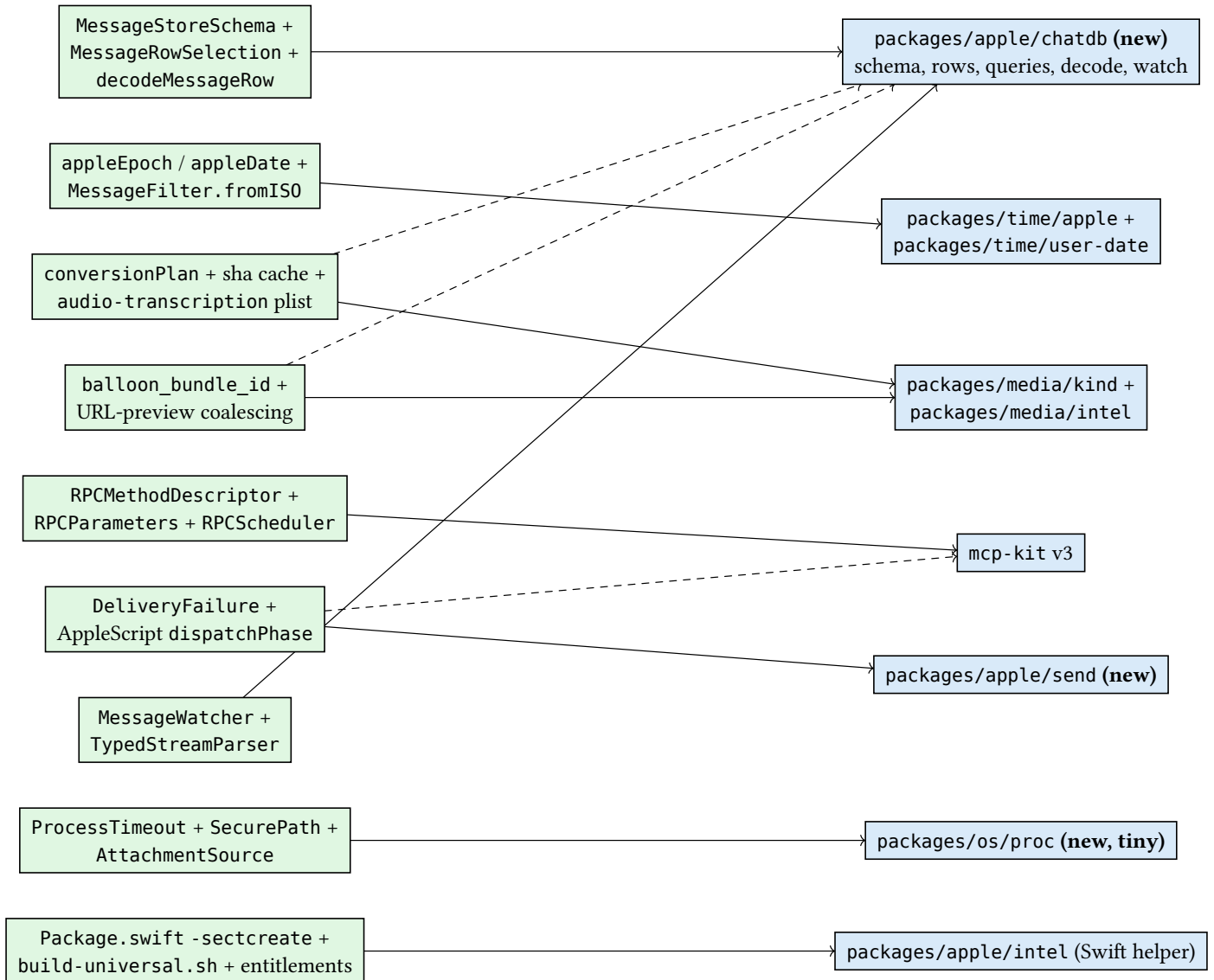


Figure 2: Mapping from openclaw/imsmsg structures (left, green) to our target packages (right, blue). Dashed edges are secondary destinations. The three packages marked **new** are proposals beyond the plan's table (apple/chatdb, apple/send, os/proc); time/*, media/*, mcp-kit and apple/intel are already in the plan.

Why a new packages/apple/chatdb. The plan's package table (time/*, media/*) removes date, MIME and filter logic from imessage-db.ts but explicitly does not rewrite it (plan item 10). The strongest structural lesson in this repo (lesson 1) is the one the plan defers. A chatdb package would be the landing place for the schema probe, row selection and decode, query objects and the watcher, and it is also what the fixture builder (lesson in testing) needs to share with the media packages. I recommend adding it to the plan as a **non-blocking** parallel track after F, because B (data-access correctness) otherwise keeps editing the same 2892-line file.

5 Reuse candidates and licence

Candidate	Assessment	Verdict
Vision OCR or Speech helper	NOT IN THEIR REPO . There is no Swift code that calls Vision, Speech, SFSpeechRecognizer or SpeechAnalyzer (search described in the summary). packages/apple/intel must be written from scratch; only its build and signing recipe can be borrowed.	No component to reuse
imsmsg rpc as a sidecar	A ready JSON-RPC-over-stdio reader/sender (imsmsg rpc, supervised as a child that exits when stdin closes, docs/rpc.md:99 @ d604a8b). Pros: mature read/watch/send semantics, typed delivery failures. Cons: version 0.15.10 pre-1.0 with fast churn (441 commits between 2025-12-05 and 2026-10-06 per git log of the clone; version in version.env:1 @ d604a8b), requires macOS 14+ and a Swift 6	Study, do not depend. Use as a decoding oracle in tests only

Candidate	Assessment	Verdict
	toolchain to build (Package.swift:6 @ d604a8b), overlaps our Rust native addon, and adds a second runtime to a package that ships as MCPB. Not a fit as a dependency.	
IMsgCore as a SwiftPM library	Reusable only from Swift. Our native helper boundary is Swift for Apple frameworks (Speech, Vision), not for chat.db, which we already read in TypeScript and Rust. Linking it would pull in PhoneNumberKit and SQLite.swift for no gain.	Skip
Small algorithms to re-implement	TypedStreamParser (63 lines), AudioMessagePreparer argv and post-check, AttachmentResolver.conversionPlan, ProcessTimeout.terminate, SecurePath.hasSymlinkComponent, AddressBook v22 query, DeliveryFailure vocabulary, the IMSG_RESULT protocol. All are short, self-contained, and described above.	Re-implement from the description; add attribution comments
Injected IMCore dylib (Sources/IMsgHelper, 7,280 lines Obj-C)	Needs SIP disabled for system-app injection, private selectors that vary by macOS release, arm64e slice, and may still be blocked by library validation (README.md:89 @ d604a8b, Sources/IMsgHelper/README.md:5 @ d604a8b). Unacceptable for a tool that reads private message data and ships to other users.	Reject. R1a may still weigh the features it enables (typing, read receipts, edit/unsend)
Build, sign and Info.plist recipe	Directly reusable pattern for packages/apple/intel (lesson 10).	Adopt the recipe

5.1 Licence verdict

openclaw/imshow is MIT, Copyright (c) 2026 Peter Steinberger (LICENSE:1-3 @ d604a8b). Our apps/imshow-mcp is also MIT, Copyright (c) 2026 George G (LICENSE:1-3 @ 91b08bb, and "license": "MIT" in package.json:96). Studying, re-implementing algorithms and even copying code are permitted. MIT's condition is that the copyright and permission notice accompany "all copies or substantial portions", so:

- **Re-implementation from the described algorithm** in TypeScript: no notice obligation in practice, but I recommend a one-line provenance comment ("approach from openclaw/imshow, MIT") at each site.
- **Verbatim or near-verbatim transliteration of non-trivial code** (for example the AppleScript source, or TypedStreamParser): add a THIRD_PARTY_NOTICES entry with the MIT text and copyright line.
- **Dependencies** of theirs (relevant only if we ever link or fork): Commander MIT, SQLite.swift MIT, PhoneNumberKit MIT (GitHub licence API, queried 2026-10-08). CSQLite returned **no SPDX licence** from the API: unknown, would need a manual check before any use.
- This is an engineering reading of the licence, not legal advice.

6 Risks

1. **Misreading the subject.** The brief assumed a media and MCP layer; there is none. Any plan item that says "do it the way openclaw/imshow does" for interpretation, cache provenance or schema dialect has no source. Mitigation: rely on R2/L/R3 for engines and on the kit work for schemas.
2. **Copying their unit assumption.** Taking appleEpoch as written would reintroduce our attach-search-date-units bug (nanoseconds applied to attachment.created_date). Mitigation: branded units and a per-column unit table.
3. **Typedstream "strict" path may misparse some layouts.** It scans for the first 01 2b. Mitigation: dual-parser differential test before switching the default.
4. **user_info coverage is unverified.** I did not read any real database. Whether the audio-transcription key is populated on gmac's macOS 15.7, and whether it agrees with the IMAudioTranscription attribute, is unknown. A count-only schema query (SELECT COUNT(*) FROM attachment WHERE user_info IS NOT NULL) would settle presence without reading content.
5. **Timeout numbers are OS-specific.** 150 s send timeout is derived for macOS 26 (Sources/IMsgCore/IMsgBridgeProtocol.swift:28-30 @ d604a8b); our host is 15.7.
6. **Upstream churn.** 441 commits since 2025-12-05 and breaking-looking features (polls, scheduled sends); anything we mirror should be mirrored as an idea, not a dependency.
7. **Package proliferation.** Proposing three new packages (apple/chatdb, apple/send, os/proc) adds workspace surface. os/proc could instead live inside media/intel until a second consumer appears.

7 Unknowns (not verified)

- Whether `SentMessageVerifier.swift` (155 lines) and `RichLinkPreparer.swift` (356 lines) contain additional transferable algorithms; I read only their callers.
- Whether `OutputModels.swift` defines Codable output structs that would count as a typed output contract.
- Whether our `change-watcher.ts` already handles a message row that precedes its `chat_message_join` row (no match for “unresolved” in the file).
- Whether `node:sqlite` can open with `mode=ro` URI semantics.
- The exact scope of `DurationParser.swift` and any relative-date support in their CLI.
- Poll decoding (`MessagePollDecoder`, `PollKeyedArchiveResolver`) uses a cycle- and depth-limited `NSKeyedArchiver` graph walk (`Sources/IMsgCore/PollKeyedArchiveResolver.swift:3-30 @ d604a8b`); it is a feature area for R1a, and I did not evaluate whether a Node bplist reader suffices for our own `payload_data` needs.
- Nothing here was run: no build of their binary, no execution against any database. All findings are from reading source at the pinned SHA.